

HW3

1. Java modifier scope: public, private, protected, default scope

Answer: public members are accessible from any class everywhere, while private members are restricted strictly to the defining class itself. protected allows access within the same package and by subclasses in other packages, whereas default (no modifier) restricts visibility exclusively to classes within the same package.

2. What is static scope

Answer: Static scope means a member belongs to the class itself rather than to any specific object instance, meaning it is loaded once into the Metaspace and shared globally across all instances.

3. How does classloader work

Answer: The ClassLoader works through a three-step process: Loading bytecode into memory, Linking (verification, preparation, and resolution), and Initialization. It strictly adheres to the Delegation Principle, where a ClassLoader always delegates the loading request to its parent before attempting to load it itself.

4. Describe the difference between unchecked and checked exceptions in Java

Answer: Checked exceptions (inheriting from Exception) are checked at compile-time and must be explicitly handled using try-catch or declared with throws. Unchecked exceptions (inheriting from RuntimeException) occur at runtime, usually representing logical coding errors, and do not enforce compile-time handling.

5. What is the difference between finally, final, and finalize in Java?

Answer: final is a keyword used to make classes uninheritable, methods unoverridable, and variables immutable. finally is a block used with try-catch to guarantee the execution of clean-up code, while finalize is a deprecated method formerly invoked by the Garbage Collector before an object is reclaimed.

6. Define try-with-resources. How can you say that it differs from an ordinary try?

Answer: Try-with-resources is a feature that automatically closes resources (like files or streams) that implement the AutoCloseable interface at the end of the statement. It differs from an ordinary try because it eliminates the need for explicit resource-closing code in a finally block and safely handles suppressed exceptions.

7. Define Runtime Exception. Describe it with the help of an example.

Answer: A Runtime Exception is an unchecked exception that reflects programming flaws or unexpected conditions during execution, such as a NullPointerException when calling a method on a null reference, which the compiler does not force you to handle.

8. What is the difference between NoClassDefFoundError and ClassNotFoundException in Java

Answer: ClassNotFoundException is a checked exception thrown at runtime when an application explicitly tries to load a class by its string name (e.g., using Class.forName()) but the class cannot

be found on the classpath. `NoClassDefFoundError` is a critical error that occurs when the JVM tries to load a class that was available during compilation but is missing at runtime during execution.

9. Why should we clean up activities such as I/O resources in the finally block?

Answer: I/O resources must be cleaned up in the finally block to ensure they are unconditionally closed even if an exception occurs, preventing severe operating system resource leaks (file descriptor exhaustion).

10. Describe `OutOfMemoryError` in exception handling.

Answer: `OutOfMemoryError` is a fatal Error (not an Exception) thrown when the JVM runs out of memory and the Garbage Collector cannot reclaim enough space, meaning it cannot be effectively resolved or recovered from within normal application exception-handling logic.

11. What is Generics in Java? What are the advantages of using Generics?

Answer: Generics allow types (classes, interfaces, and methods) to be parameterized, providing a way to reuse code with different data types. The primary advantages are ensuring compile-time type safety and eliminating the need for explicit, error-prone manual type-casting.

12. How does Generics work in Java? What is type erasure?

Answer: Java Generics are implemented via Type Erasure, meaning the compiler enforces type constraints during compilation but removes all generic type parameters before generating the bytecode. At runtime, the generic types are replaced by their bounded types or `Object`, making generics backward-compatible with older Java versions.

13. What is the difference between `List<? extends T>` and `List<? super T>`?

Answer: `List<? extends T>` establishes an upper bound, meaning it represents a list of `T` or any subclass of `T`, which is safe for reading elements as type `T` (Producer Extends). `List<? super T>` establishes a lower bound, representing a list of `T` or any superclass of `T`, making it safe for writing elements into the list (Consumer Super).

14. What is Optional class?

Answer: The `Optional` class is a container object introduced in Java 8 used to represent the presence or absence of a non-null value, effectively minimizing the occurrence of defensive null checks and `NullPointerExceptions`.

15. What is OOP

Answer: OOP (Object-Oriented Programming) is a programming paradigm based on the concept of "objects," which bundle data and behaviors together. Its foundation rests on four core pillars: Encapsulation (hiding internal state), Inheritance (reusing code), Polymorphism (one interface, multiple behaviors), and Abstraction (hiding implementation details).

HW 4

1. What is functional interface

Answer: A functional interface is an interface that contains exactly one abstract method, allowing it to be used as the target type for Lambda expressions or method references (often annotated with `@FunctionalInterface`).

2. What is default method

Answer: A default method is an instance method defined inside an interface using the `default` keyword, which provides a concrete implementation and allows new capabilities to be added to existing interfaces without breaking backward compatibility for old implementing classes.

3. What is the difference between Predicate, Supplier, Consumer, Function?

Answer: `Predicate<T>` accepts one argument and returns a boolean. `Supplier<T>` takes no arguments but supplies an output of type `T`. `Consumer<T>` accepts one argument of type `T` and returns no result (`void`). `Function<T, R>` accepts an input of type `T` and transforms it into an output of type `R`.

4. What is method reference

Answer: A method reference (using the `::` syntax) is a shorthand, highly readable way to write a Lambda expression that simply calls an existing method by its name.

5. What is CompletableFuture

Answer: `CompletableFuture` is an advanced class introduced in Java 8 that implements the `Future` and `CompletionStage` interfaces, enabling powerful non-blocking, asynchronous programming by allowing tasks to be chained, combined, and handled through functional callbacks.

6. Default keyword vs Java default scope

Answer: The `default` keyword is an explicit syntax modifier used to define concrete methods inside interfaces or cases within a `switch` statement. Java default scope (`package-private`), however, is an implicit access modifier achieved by omitting any visibility keyword, which limits access to the same package.

7. Intermediate operation vs terminal operation in Stream API

Answer: Intermediate operations (like `map` and `filter`) transform a stream into another stream and are lazy, meaning they do not execute until a terminal operation is called. Terminal operations (like `collect` and `forEach`) traverse the stream, trigger the actual pipeline execution, and produce a final result or side effect, after which the stream is consumed and closed.

8. Stream API: `map()` vs `flatMap()`

Answer: `map()` is a one-to-one mapping operation that transforms each element of a stream individually into another object. `flatMap()` is a one-to-many flattening operation that transforms each element into a stream of elements and then merges all these sub-streams into a single, unified continuous stream.